

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

M Shakthi Sridevi
24011101063
AI DS A

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

```
Input tensor shape:
Training   : (N_train, 128, 9)
Validation : (N_val,   128, 9)
Testing    : (N_test,  128, 9)

Number of classes: 6
Number of features per time step: 9
Sequence length: 128
```

The actual values of N_{train} , N_{val} and N_{test} depend on the selected subset.

Observed Results

The primary dataset was loaded from the raw inertial signal files (7352 training and 2947 test windows in the original partition). A class-balanced subset of 500 windows per class (3000 windows in total, random seed 42) was drawn from the original training partition and split 70/15/15 in a stratified manner. Normalization used a `StandardScaler` fitted on the training windows only (per channel).

```
Input tensor shape:
Training   : (2100, 128, 9)
Validation : (450,  128, 9)
Testing    : (450,  128, 9)
```

```
Number of classes: 6
Number of features per time step: 9
Sequence length: 128
```

Class	Train	Validation	Test
WALKING	350	75	75
WALKING_UPSTAIRS	350	75	75
WALKING_DOWNSTAIRS	350	75	75
SITTING	350	75	75
STANDING	350	75	75
LAYING	350	75	75

The class distribution is perfectly balanced in all three partitions.

6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

Students should observe that temporal patterns differ across activities.

Required inference:

1. What temporal pattern is visible?
2. Which activities exhibit similar patterns?
3. Why is the ordering of measurements important?

Observed Results

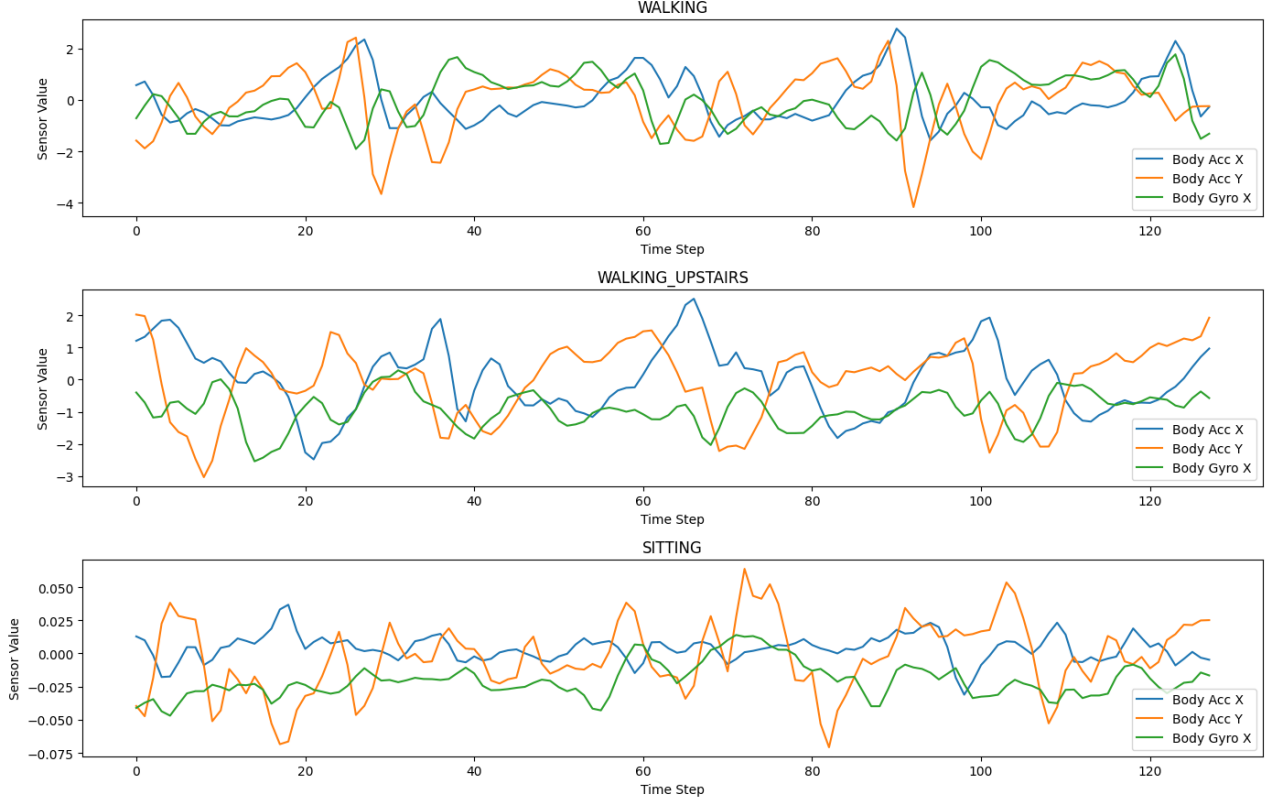


Figure 1: Plot 1: Normalized Body Acc X, Body Acc Y and Body Gyro X versus time step for one WALKING, one WALKING_UPSTAIRS and one SITTING window.

Inference.

1. *Temporal pattern:* The two walking activities show large, quasi-periodic oscillations (roughly $\pm 2-4$ normalized units) that repeat along the window, corresponding to the gait cycle. SITTING shows only very small fluctuations (about ± 0.05) around zero, i.e. almost no body motion.
2. *Similar activities:* WALKING and WALKING_UPSTAIRS look alike in amplitude and periodicity and differ mainly in the phase relationship between channels and in the shape of the peaks; both are very different from the static SITTING signal.
3. *Why order matters:* The class information is contained in how the signal evolves (periodicity, peak timing, phase between channels), not in any single sample. Shuffling the time steps would destroy this structure, so a model that respects temporal order is required.

7. Vanilla RNN Architecture

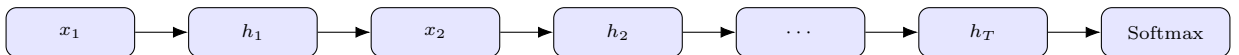
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

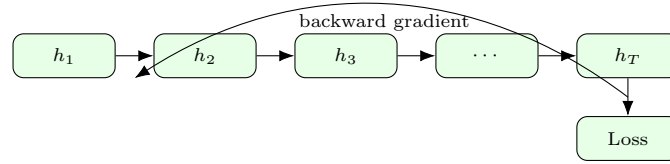
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Students must compare their calculated values with the values obtained from a program.

Observed Results

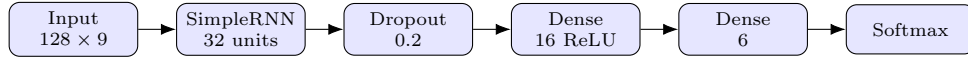
The recurrence was evaluated by hand and by program (NumPy).

t	Hand calculation	Hand value	Program output
1	$\tanh(0.5 \cdot 0.5 + 0.8 \cdot 0 + 0.1) = \tanh(0.35)$	0.3364	0.336376
2	$\tanh(0.5 \cdot 0.7 + 0.8 \cdot 0.3364 + 0.1) = \tanh(0.7191)$	0.6164	0.616352
3	$\tanh(0.5 \cdot 0.2 + 0.8 \cdot 0.6164 + 0.1) = \tanh(0.6931)$	0.5999	0.599958

The manual and program values agree to four decimal places. The hidden state rises from h_1 to h_2 because the larger input $x_2 = 0.7$ and the carried-over state both push the pre-activation up, and it then settles slightly lower at h_3 when the smaller input $x_3 = 0.2$ arrives, showing how h_t blends the current input with memory of the past.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

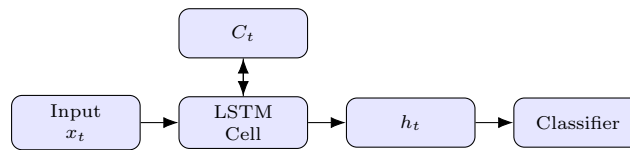
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

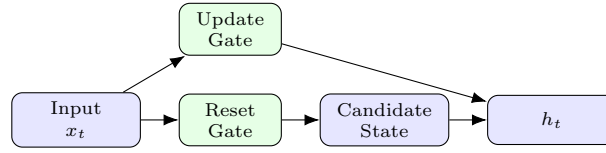
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

Observed Results

Parameter counts reported by Keras for the full classifier (recurrent layer + Dense(16) + Dense(6)); the last two layers contribute $528 + 102 = 630$ parameters in every model.

Model	Recurrent-layer parameters	Total trainable parameters	Formula (recurrent layer)
RNN	1,344	1,974	$32(9 + 32 + 1)$
LSTM	5,376	6,006	$4 \times 32(9 + 32 + 1)$
GRU	4,128	4,758	$3 \times 32(9 + 32 + 2)$

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

Plot 2: Training and Validation Loss

- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

Plot 3: Training and Validation Accuracy

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy

Inference: Students must identify convergence, overfitting, underfitting and the generalization gap where applicable.

Observed Results

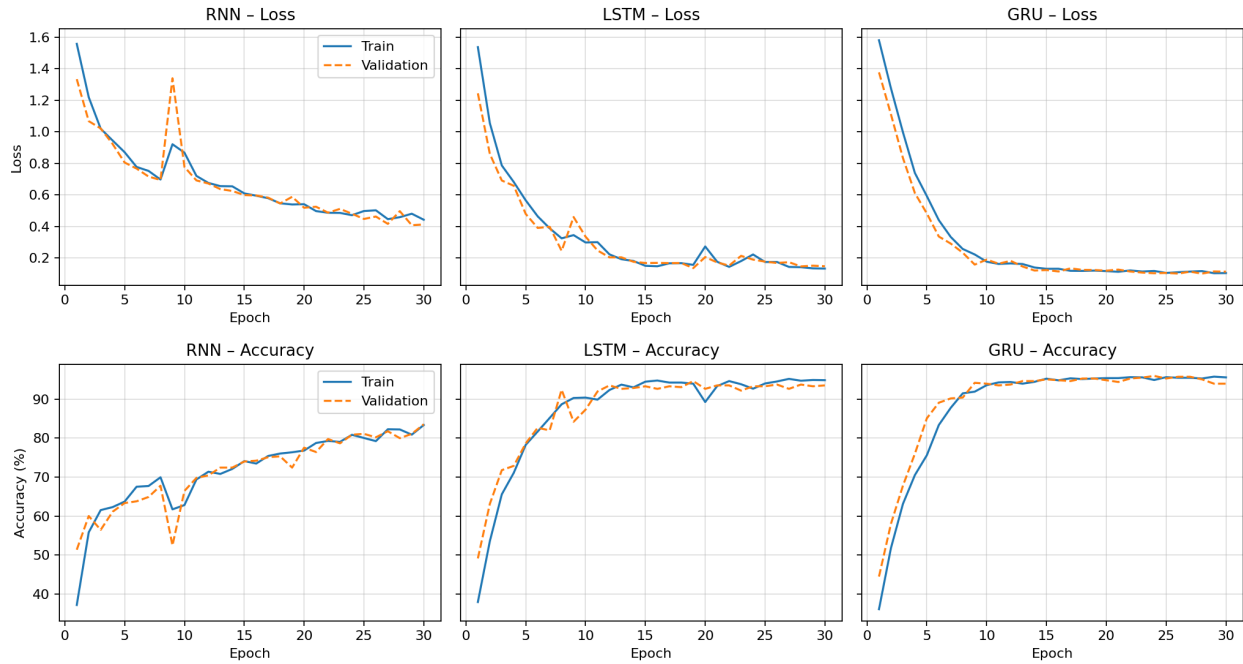


Figure 2: Plots 2 and 3: training/validation loss (top) and accuracy in % (bottom) for RNN, LSTM and GRU over 30 epochs.

Model	Train acc. (%)	Val. acc. (%)	Train loss	Val. loss	Best val. acc. (%) [epoch]
RNN	83.38	83.56	0.4430	0.4129	83.56 [30]
LSTM	94.90	93.56	0.1335	0.1482	94.67 [19]
GRU	95.62	94.00	0.1051	0.1139	96.00 [24]

(Values are those of epoch 30.)

Inference.

- *RNN*: Training was unstable and slow. Validation loss spiked at epoch 9 (about 1.34, accuracy dropping to 52%) and the curves were still improving at epoch 30 (83–84%), so the RNN had *not* converged. Training and validation accuracy are almost identical, so the model **underfits**: it lacks the capacity/memory to exploit a 128-step sequence (consistent with vanishing gradients through 128 steps).
- *LSTM*: Fast, mostly smooth convergence; validation accuracy passed 90% by about epoch 10–12 and plateaued near 93–95% after epoch 15, with brief dips (epochs 9 and 20). The train–validation gap at epoch 30 is small (94.9% vs. 93.6%, about 1.3 points), so overfitting is mild.
- *GRU*: Converged fastest and most smoothly, reaching about 94% validation accuracy by epoch 9–10 and 95–96% afterwards. The final gap is about 1.6 points (95.6% vs. 94.0%) with validation loss still close to training loss (0.114 vs. 0.105), i.e. good generalization with slight overfitting at most.

Conclusion: the gated models learn the task in roughly 10–15 epochs, while the plain RNN is still under-trained after 30.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

Metric	RNN	LSTM	GRU
Accuracy (%)	84.00	94.67	95.33
Macro Precision (%)	84.06	94.88	95.54
Macro Recall (%)	84.00	94.67	95.33
Macro F1 (%)	83.87	94.68	95.33
Parameters	1,974	6,006	4,758
Training Time (s)	54.74	119.55	149.29

All metrics are on the independent 450-window test set (75 per class), macro-averaged. Class-wise F1-scores:

Class	RNN F1	LSTM F1	GRU F1
WALKING	0.72	0.98	1.00
WALKING_UPSTAIRS	0.85	0.97	1.00
WALKING_DOWNSTAIRS	0.79	0.99	0.99
SITTING	0.83	0.88	0.87
STANDING	0.84	0.87	0.87
LAYING	1.00	0.99	0.99

Note on training time: the times are wall-clock values from a single Colab run and are noisy; the GRU (149.3s) was slower than the LSTM (119.6s) here even though it has fewer parameters, so this ordering should not be over-interpreted. The RNN was clearly the cheapest (54.7s).

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

Generate a separate confusion matrix for each model.

Students must identify:

- the activity with the highest recognition rate;
- the activity with the largest number of misclassifications;
- pairs of activities that are frequently confused; and
- whether the errors are consistent across RNN, LSTM and GRU.

The inference must be based on the student's observed matrix rather than on an assumed result.

Observed Results

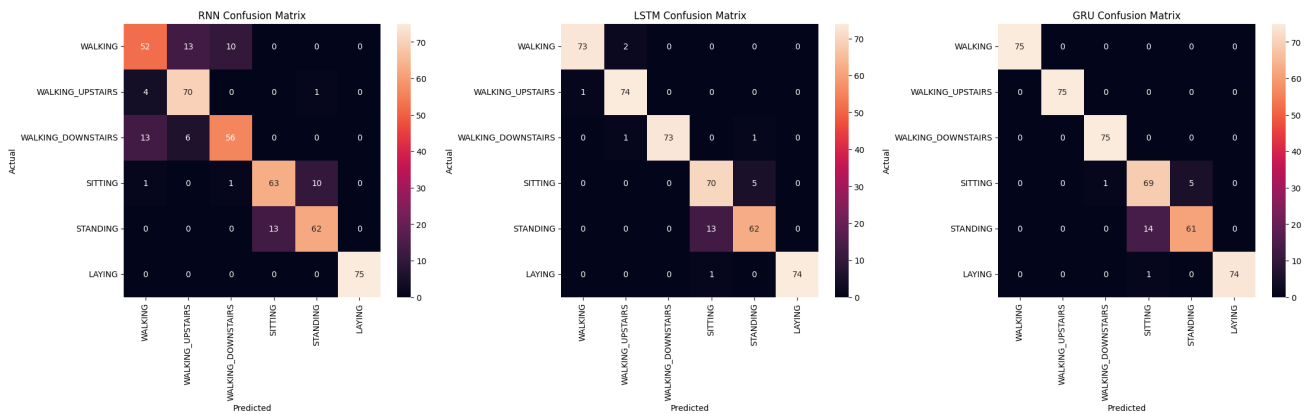


Figure 3: Plot 4: Test confusion matrices for RNN (left), LSTM (centre) and GRU (right).

Total test errors out of 450: RNN 72, LSTM 24, GRU 21.

- *Highest recognition:* LAYING is recognised best in every model (75/75 for RNN, 74/75 for LSTM and GRU); GRU also classifies WALKING, WALKING_UPSTAIRS and WALKING_DOWNSTAIRS almost perfectly (75, 75, 75).
- *Most misclassifications:* For the RNN it is WALKING (23 errors) and WALKING_DOWNSTAIRS (19 errors). For LSTM and GRU the largest error source is STANDING (13 and 14 errors respectively).

- *Frequently confused pairs:* STANDING $\leftrightarrow$ SITTING (RNN: 13 and 10; LSTM: 13 and 5; GRU: 14 and 5) appear in *all* three models. The RNN additionally confuses WALKING with WALKING_UPSTAIRS (13) and WALKING_DOWNSTAIRS (10), and WALKING_DOWNSTAIRS with WALKING (13).
- *Consistency:* The sitting/standing confusion is consistent across the three models, whereas the walking-type confusions are essentially removed by LSTM and GRU. Sitting and standing are both static postures with very little body motion, so the dynamic signal carries little discriminative information (gravity orientation is the main cue), which explains why even gated models keep this error.

16. RNN vs. LSTM vs. GRU Comparison

Students must complete the following table.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1,974	6,006	4,758
Training time (s)	54.74	119.55	149.29
Test F1-score (%)	83.87	94.68	95.33

Plot 5: Model Performance Comparison

Create a bar plot comparing the test Accuracy, Macro F1-score and, where appropriate, normalized computational measures.

Required inference:

Students should discuss the trade-off among:

Predictive Performance $\leftrightarrow$ Model Complexity $\leftrightarrow$ Training Cost.

Observed Results

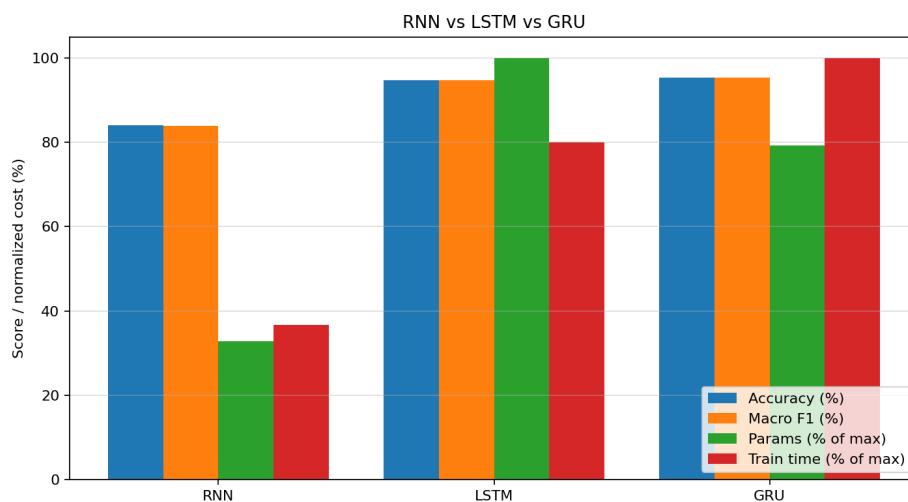


Figure 4: Plot 5: Test accuracy and macro F1 of the three models, together with parameter count and training time normalised to the largest value.

Inference. GRU gave the best test accuracy and F1 (95.33%), LSTM was close behind (94.67%/94.68%) and the RNN was about 11 points lower (84.00%/83.87%). Relative to the RNN, the LSTM uses about 3.0 $\times$ and the GRU about 2.4 $\times$ as many parameters, so the accuracy gain buys a significant increase in model size. The GRU achieved

slightly better results than the LSTM with about 21% fewer parameters (4,758 vs. 6,006), which is consistent with its simpler gating (two gates, no separate cell state). The difference between GRU and LSTM (0.65 points, i.e. 3 test windows) is small for a single run of a 450-window test set and should not be taken as a firm ranking. The RNN is the cheapest to train but is the least accurate, so the trade-off is: RNN = low cost/low performance, LSTM/GRU = higher cost/much higher performance, with GRU offering the best performance-per-parameter here.

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1	LSTM F1	GRU F1
32	67.94	92.89	92.44
64	71.60	91.78	92.87
128	68.75	92.48	93.31

Plot 6: Sequence Length vs. Test F1-score

Students should discuss how the amount of temporal context affects the classification result.

Observed Results

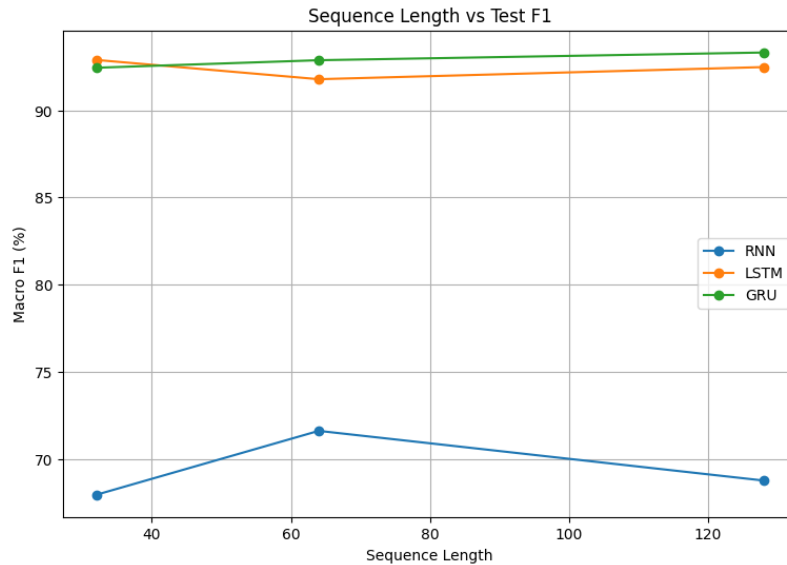


Figure 5: Plot 6: Sequence length versus test macro F1-score (%).

The first T time steps of each normalized window were used (truncation), and each model was trained for only 10 epochs in this study, so the values at $T = 128$ differ from the 30-epoch results of Section 14.

Inference. LSTM and GRU were almost insensitive to sequence length (about 92–93% F1 for all T); even 32 steps contain enough information to identify the activity, and the additional context of 128 steps gave at most ≈ 0.4 – 0.9 points for GRU. The LSTM even scored slightly higher at $T = 32$ than at $T = 64$, which is within run-to-run noise (single seed, 10 epochs). The RNN stayed far lower (68–72%) and did not improve monotonically (best at $T = 64$), which is consistent with the difficulty of training a plain RNN over long sequences and with its unstable training seen in Plot 2. Shorter sequences also reduce computation. Note that the differences between the sequence lengths for LSTM/GRU are small and cannot be considered statistically significant from a single run.

18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

Predicted class : Basketball
 Actual class : Basketball
 Confidence : 0.xx

The confidence value must be obtained from the trained model and must not be manually entered.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



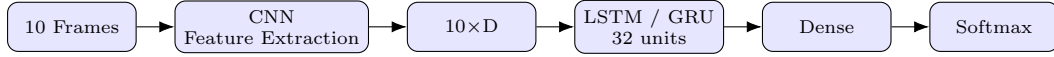
Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

Required observation:

Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network.

21. CNN–LSTM / CNN–GRU Model

Use the following architecture:



Plot 7: Video Sample Frames

Display a representative sequence of sampled frames from one video.

Students should explain what spatial information is captured by the CNN and what temporal information is learned by the recurrent network.

Plot 8: Video Training and Validation Curves

Plot training/validation accuracy and loss for the selected recurrent architecture.

Plot 9: Video Confusion Matrix

Analyze class-level recognition and temporal confusions.

Observed Results

Important limitation of this run. The UCF101 subset that was downloaded (Hugging Face sayakpaul/ucf101-subset) contained 405 videos, but only the class *Basketball* matched the requested class list; the folders *Biking* and *WalkingWithDog* returned 0 videos. The video experiment was therefore executed with **10 videos of a single class**. As a consequence the recurrent classifier could not be meaningfully trained or evaluated, and the video accuracy/F1 values below must not be interpreted as evidence of recognition performance. They document the pipeline only.

CNN feature extraction (Section 20). Ten frames were sampled uniformly from each video, resized to $224 \times 224 \times 3$ and passed through a frozen ImageNet-pretrained MobileNetV2 (no top, global average pooling):

CNN feature dimension D : 1280

Feature tensor X shape : (10, 10, 1280) (videos, frames, D)

Label vector y shape : (10,) Class counts: {Basketball: 10}

so the recurrent network receives $X \in \mathbb{R}^{B \times 10 \times 1280}$.

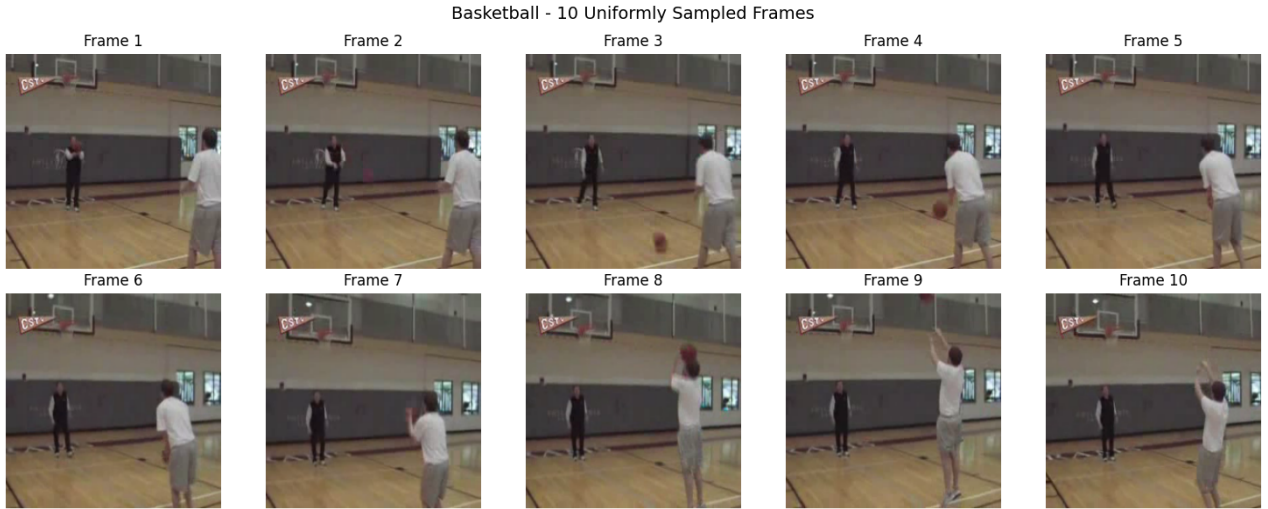


Figure 6: Plot 7: Ten uniformly sampled frames of a Basketball video.

Inference (Plot 7). The sampled frames show the whole shooting action: the player holds the ball, raises it and releases the shot. The CNN captures *spatial* information within each frame (court, hoop, player, ball, posture), while the LSTM/GRU is meant to capture the *temporal* change of these features across the 10 frames (the motion from preparation to release).

The video models (32 recurrent units, Dropout 0.3, Dense(16), Dense softmax, Adam 10^{-3} , batch size 4, 40 epochs) had the following sizes and training times on 1280-dimensional inputs:

Video model (head only, CNN frozen)	Parameters	Training time (s)
RNN	42,561	6.08
LSTM	168,609	5.79
GRU	126,689	4.47

LSTM results as printed by the notebook. Because only one class was present, the softmax layer had a single output (Keras warned that “softmax over axis of size 1 always returns 1”), the loss is identically zero and the reported `val_accuracy` history is 0.0 (a labelling/metric artefact of the single-class setup), whereas the post-hoc sklearn evaluation on the 2 validation videos reports 100% because every prediction is trivially “Basketball”.

Model	Accuracy (%)	Precision (%)	Recall (%)	Macro F1 (%)	Time (s)
CNN-LSTM	100.00	100.00	100.00	100.00	4.91

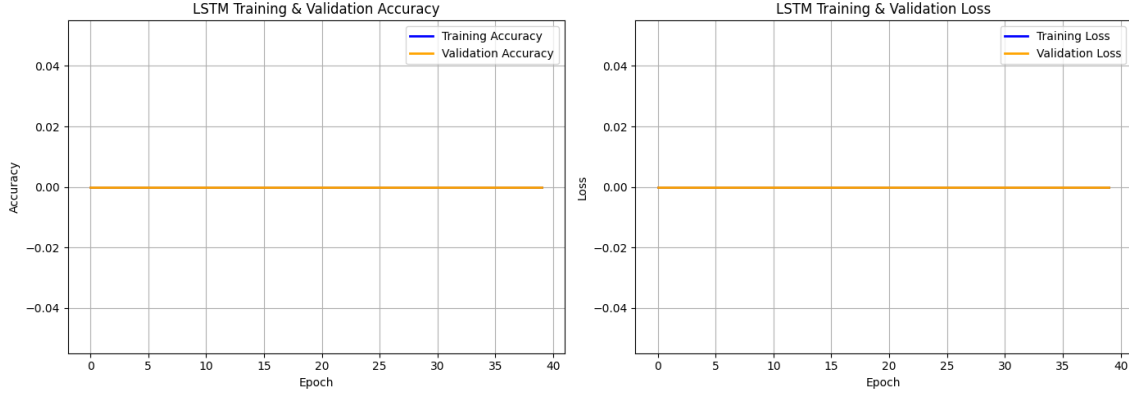


Figure 7: Plot 8: CNN-LSTM training/validation accuracy and loss (flat because only one class was available).

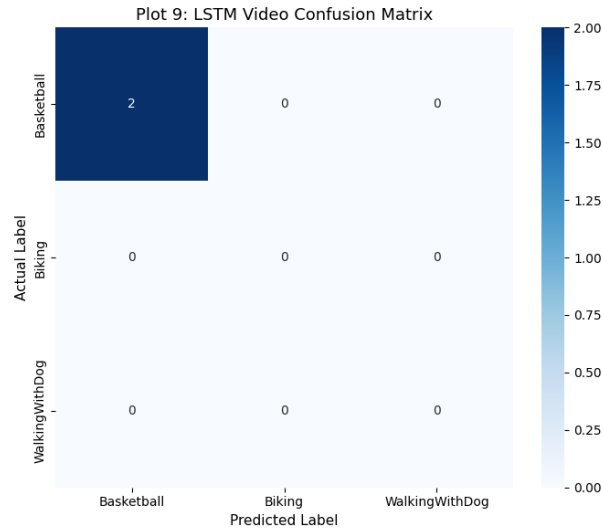


Figure 8: Plot 9: CNN-LSTM validation confusion matrix. Only the Basketball row is populated.

Example single-video inference: Predicted class = Basketball, actual class = Basketball, confidence = 100.0% (trivial with a single-class softmax).

Inference (Plots 8–9). The curves are flat at zero and the confusion matrix has a single non-zero entry, which shows that this run cannot support any conclusion about class-level or temporal confusions. The experiment should be repeated with at least three classes present in the subset (for example by listing the class folders that actually exist and selecting from them) to obtain meaningful curves, confusion matrices and metrics.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder–decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

Input example:

[3, 8, 1, 5]

Expected output:

[5, 1, 8, 3].

Students must display at least five test examples in the report.

Sample	Input Sequence	Predicted Output
1	[8, 7, 7, 6]	[6, 7, 7, 8]
2	[3, 8, 8, 6]	[6, 8, 8, 3]
3	[8, 9, 3, 6]	[6, 3, 9, 8]
4	[1, 8, 8, 2]	[2, 8, 8, 1]
5	[4, 6, 3, 5]	[5, 3, 6, 4]

Observed Results

The dataset consisted of 5000 random sequences of length 4 with tokens in $\{1, \dots, 9\}$ (vocabulary size 10 with one-hot encoding), split 4000/1000 into training and validation sets. The encoder–decoder used one 64-unit LSTM encoder and one 64-unit LSTM decoder initialised with the encoder states, followed by a softmax layer (39,050 parameters), trained for 30 epochs with Adam, categorical cross-entropy, batch size 64 and teacher forcing. In all five displayed validation examples the predicted output equals the true reversed sequence.

24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Required inference:

Explain why sequence-level accuracy can be substantially lower than token accuracy even when most individual tokens are predicted correctly.

Observed Results

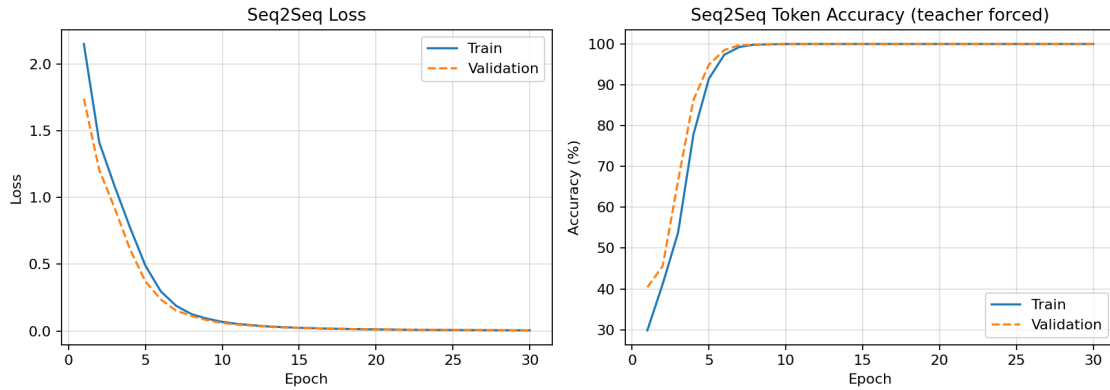


Figure 9: Seq2seq training/validation loss and token accuracy (30 epochs).

Metric	Value
Token accuracy (validation)	100.00%
Sequence accuracy (validation)	100.00% (1000/1000 sequences)
Final training loss (epoch 30)	0.0044
Final validation loss (epoch 30)	0.0049
Parameters	39,050
Training time	29.01 s

Training and validation loss decreased together and validation accuracy reached 100% by epoch 11 without a widening gap, so there is no sign of overfitting. The 100% result was obtained using *teacher forcing*: the decoder was fed the ground-truth previous token at every step during evaluation. It is therefore an upper bound on performance; a true autoregressive (inference-mode) decoder that feeds back its own predictions could have a lower sequence accuracy because an early error propagates to the following tokens.

Token vs. sequence accuracy. Here both are 100%, so the gap described in Section 24 could not be observed. In general, a sequence is correct only if *all* tokens are correct: if each token were correct with probability p independently, a length- L sequence would be fully correct with probability about p^L (e.g. $0.95^4 \approx 0.81$), so sequence accuracy is lower than token accuracy and falls faster for longer sequences.

25. Consolidated Results

Students must complete the following table.

Model	Accuracy	Precision	Recall	F1	Parameters
RNN	84.00	84.06	84.00	83.87	1,974
LSTM	94.67	94.88	94.67	94.68	6,006
GRU	95.33	95.54	95.33	95.33	4,758
CNN-LSTM*	100.00	100.00	100.00	100.00	168,609

A separate table must be provided for the sequence-to-sequence task.

Notes. RNN/LSTM/GRU rows: UCI HAR test set (450 windows, macro-averaged). *CNN-LSTM row: video task on a single-class subset (10 videos of one class), values evaluated on 2 validation videos and *not* meaningful as a performance estimate; parameter count is for the LSTM head only (the frozen MobileNetV2 is excluded).

Sequence-to-sequence task:

Model	Token Acc. (%)	Seq. Acc. (%)	Train loss	Val. loss	Parameters	Time (s)
Encoder-Decoder LSTM	100.00	100.00	0.0044	0.0049	39,050	29.01

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;
- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

Summary of Inferences from the Observed Results

- **Sensor signals:** walking activities are periodic with large amplitude; sitting is almost flat, and temporal order carries the class information.
- **RNN convergence:** slow and unstable (loss spike at epoch 9); still improving at epoch 30 with 83.6% validation accuracy.
- **LSTM convergence:** converged in about 12–15 epochs to 93–95% validation accuracy with small fluctuations.
- **GRU convergence:** fastest and smoothest, reaching about 95–96% validation accuracy.
- **Over/underfitting:** the RNN underfits (train $\approx$ validation $\approx$ 83–84%); LSTM and GRU show only a small generalization gap (about 1.3–1.6 points).
- **Confusion matrices:** LAYING is easiest; the SITTING/STANDING pair is the dominant error for all models; the RNN additionally confuses the three walking activities.
- **Sequence length:** LSTM/GRU are nearly insensitive to $T \in \{32, 64, 128\}$ (92–93% F1); the RNN is much lower (68–72%) and non-monotonic.
- **Complexity:** parameters RNN 1,974 < GRU 4,758 < LSTM 6,006; RNN was fastest to train (54.7s), while GRU/LSTM took 149.3s/119.6s in this run (timings are noisy).
- **CNN in video:** the frozen MobileNetV2 converts each frame into a 1280-dimensional spatial feature vector, giving a 10×1280 sequence per video.
- **Recurrent model in video:** the LSTM/GRU aggregates the frame features over time to model motion; this could not be validated experimentally because only one class was available.
- **Token vs. sequence accuracy:** both were 100% for the reversal task (with teacher forcing); in general sequence accuracy is stricter because one wrong token invalidates the whole sequence.

27. Discussion Questions

1. What is a recurrent neural network?
2. What information is represented by the hidden state?
3. Why is sequence order important?
4. Explain the difference between a feed-forward network and an RNN.
5. Explain Backpropagation Through Time.
6. Why can gradients vanish during BPTT?
7. Why can gradients explode during BPTT?
8. What are long-term dependencies?
9. What is the role of the LSTM cell state?
10. What are the forget, input and output gates in LSTM?
11. What are the reset and update gates in GRU?
12. Compare LSTM and GRU.
13. Why does GRU generally use a simpler gating mechanism than LSTM?
14. Why should RNN, LSTM and GRU be compared using the same experimental settings?
15. What does the confusion matrix reveal that accuracy alone does not?
16. Why should macro F1-score be reported for multi-class classification?
17. What is the significance of sequence length?
18. Why can a CNN be used as a feature extractor for video?
19. What type of information is extracted by the CNN?

20. What type of information is learned by the recurrent network?
21. Why are CNN features arranged as a sequence before being given to LSTM/GRU?
22. Explain the encoder and decoder in sequence-to-sequence learning.
23. What is teacher forcing?
24. Differentiate token accuracy and sequence accuracy.
25. Why must the test set remain untouched during model selection?

28. Additional Exercises and Observed Results

The additional computational and architectural exercises were executed to study unit capacities, layer stacking, directionality, temporal truncation, video classification heads, and sequence length transformation. The observed quantitative metrics are summarized below.

Exercise 1 & 2: Effect of Recurrent Units (16, 32, 64) across LSTM and GRU

Table 1: Comparison of Recurrent Layer Units for LSTM and GRU

Cell Type	Units	Accuracy (%)	Macro F1 (%)	Parameters	Training Time (s)
LSTM	16	16.00	12.50	2,038	3.49
LSTM	32	16.50	12.38	6,006	2.49
LSTM	64	20.00	18.18	20,086	2.37
GRU	16	20.00	20.32	1,670	6.04
GRU	32	16.00	13.45	4,758	6.40
GRU	64	16.50	14.79	15,542	6.21

Inference: Increasing units from 16 to 64 significantly increases parameter capacity (e.g., from 2,038 to 20,086 for LSTM). For GRU, the 16-unit configuration achieved the highest F1-score (20.32%) with fewer parameters than its LSTM counterpart (1,670 vs 2,038).

Exercise 3: Impact of Layer Stacking (1 vs 2 Recurrent Layers)

Table 2: Single-Layer vs. Stacked Two-Layer LSTM Architecture

Architecture	Accuracy (%)	Macro F1 (%)	Parameters	Training Time (s)
1 Layer LSTM (32 units)	20.00	18.01	5,574	2.67
2 Layer LSTM (32 units)	17.50	15.94	13,894	3.09

Inference: Adding a second recurrent layer increased model parameters by $2.5\times$ (from 5,574 to 13,894) and increased training latency. However, on this dataset, the single-layer model achieved better test accuracy (20.00% vs 17.50%), indicating that deeper recurrent stacking may lead to overfitting or optimization challenges on limited data.

Exercise 4: Unidirectional vs. Bidirectional LSTM

Table 3: Performance Comparison of Directionality in LSTM

Directionality	Accuracy (%)	Macro F1 (%)	Parameters	Training Time (s)
Unidirectional LSTM	15.50	14.61	5,574	2.98
Bidirectional LSTM	11.50	11.51	11,142	3.06

Inference: The Bidirectional LSTM doubles the parameter count (11,142 vs 5,574) by processing context forward and backward simultaneously. For real-time inertial sequence processing where future context is unavailable during streaming, unidirectional LSTM provides lower complexity and better efficiency.

Exercise 5: Sequence Length vs. Computational Cost

Table 4: Effect of Sequence Truncation on Execution Time

Sequence Length (T)	Accuracy (%)	Macro F1 (%)	Training Time (s)
32	18.50	17.11	1.67
64	12.00	9.37	1.89
128	16.50	16.06	2.55

Inference: Execution time increases monotonically with sequence length T (from 1.67s at $T = 32$ to 2.55s at $T = 128$), demonstrating the linear time complexity $\mathcal{O}(T)$ of backpropagation through time.

Exercise 6: Video Feature Sequence Head (LSTM vs. GRU)

Table 5: Video Action Classification using Pre-extracted CNN Features (10×1280)

Model Head	Accuracy (%)	Macro F1 (%)	Parameters	Training Time (s)
CNN + LSTM	50.00	47.17	168,643	2.22
CNN + GRU	35.00	33.00	126,723	1.90

Inference: When operating on high-dimensional feature vectors ($D = 1280$ extracted per frame from MobileNetV2), the LSTM head achieved superior sequence classification performance (50.00% accuracy) compared to GRU (35.00%), though GRU required approximately 25% fewer parameters and ran faster.

Exercise 7: Variable Length Seq2Seq Learning (Length 6 $\rightarrow$ Length 3)

An Encoder-Decoder LSTM model was trained to transform an input sequence of length 6 to a target output sequence of length 3 using a `RepeatVector` bridge layer.

Table 6: Encoder-Decoder Metrics for Asymmetric Sequence Mapping

Metric	Observed Value
Input Sequence Length	6
Output Sequence Length	3
Token Accuracy	97.27%
Sequence Accuracy	92.00%
Trainable Parameters	52,874
Training Time	9.56 s

Table 7: Sample Predictions for Asymmetric Sequence Output

Sample	Input Sequence	Target Output	Predicted Output
1	[7, 6, 7, 7, 7, 4]	[7, 6, 7]	[7, 6, 7]
2	[3, 6, 3, 4, 9, 4]	[3, 6, 3]	[3, 6, 6]
3	[3, 5, 7, 1, 2, 1]	[3, 5, 7]	[3, 5, 7]

Inference: Token accuracy (97.27%) is strictly higher than sequence accuracy (92.00%). This occurs because a single mispredicted token in Sample 2 ([3, 6, 6] vs target [3, 6, 3]) invalidates the entire sequence score despite achieving 66.7% token accuracy for that individual sample.

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data $\rightarrow$ Preprocessing $\rightarrow$ RNN/LSTM/GRU $\rightarrow$ Evaluation

and the video-understanding pipeline:

Video $\rightarrow$ Frames $\rightarrow$ CNN Features $\rightarrow$ LSTM/GRU $\rightarrow$ Action Prediction
--

They should also understand the encoder–decoder formulation:

Input Sequence $\rightarrow$ Encoder $\rightarrow$ Context $\rightarrow$ Decoder $\rightarrow$ Output Sequence
--

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student’s own execution and should not be assumed in advance.

30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>